

FinPilot System Design Document

Status: Design Review Phase
Date: March 31, 2026
Source of Truth: PROJECT_CONTEXT.md

1. RECOMMENDED TECH STACK

Backend

Component	Recommendation	Alternative	Rationale
Runtime	Node.js (v18+)	Python/Flask	JavaScript ecosystem for shared types with frontend; strong TS support
Framework	Express.js	Fastify, NestJS	Lightweight, familiar, production-proven for financial MVP
Language	TypeScript	JavaScript	Type safety critical for financial calculations; reduces runtime errors
ORM	Prisma	TypeORM, Sequelize	Type-safe schema, excellent migrations, great DX; auto-generated client
Database	PostgreSQL	MySQL, SQLite	ACID compliance for financial txns; strong JSON support for settings; mature
Validation	Zod	Joi, yup	Infers TypeScript types; better developer experience; smaller bundle
Authentication	JWT + httpOnly cookies	Sessions	Stateless (scales); secure storage; CSRF protection
Password Hashing	bcrypt	argon2	Standard library, well-tested, sufficient for MVP
CSV Processing	csv-parser + stream	papaparse	Fast, stream-based for large files; memory efficient
PDF Generation	pdfkit	jsPDF, ReportLab	Good quality, simple API, file-based output
Testing	Jest + Supertest	Mocha + Chai	Industry standard; built-in mocking; assertion library
Linting	ESLint + Prettier	TSLint	Standard tooling; easier onboarding

Frontend

Component	Recommendation	Alternative	Rationale
-----------	----------------	-------------	-----------

Framework	React 18+	Vue 3, Svelte	Large ecosystem; financial apps use React; team familiarity likely
Language	TypeScript	JavaScript	Type safety for calculations; consistency with backend
Build Tool	Vite	Create React App, Next.js	Fast dev server; smaller bundle; modern tooling
Styling	Tailwind CSS	Material-UI, Styled Components	Utility-first for rapid styling; smaller bundle; financial dashboards common in Tailwind
State Management	Zustand	Redux Toolkit, Jotai	Minimal boilerplate; easy to learn; scales for MVP needs
Server State	TanStack Query (React Query)	SWR, Zustand + Axios	Excellent caching; automatic refetch; developer tools
Routing	React Router v6	TanStack Router	Nested routes; industry standard; good TypeScript support
Charts	Recharts	Chart.js, ECharts	React-native; composable; clean API
Data Grid	TanStack Table (headless)	AG Grid, DataGridPro	Headless for flexibility; non-opinionated; free tier sufficient
Date Handling	date-fns	Day.js, moment	Tree-shakeable; modern; smaller than Moment
HTTP Client	Axios	Fetch API	Interceptors for auth token refresh; request/response transformation
Forms	React Hook Form + Zod	Formik, React Final Form	Minimal re-renders; Zod for shared validation schemas
UI Components	Headless UI + Tailwind	shadcn/ui, Daisy UI	Full control; accessible; consistent design system
Testing	Vitest + React Testing Library	Jest + RTL	Vitest faster for Vite; RTL follows best practices

DevOps & Infrastructure

Component	Recommendation	Alternative	Rationale
Containerization	Docker	Podman	Standard; easy deployment (though optional for MVP)
Package Management	pnpm	npm, yarn	Fast; efficient; monorepo-friendly disk usage
Monorepo Tool	pnpm workspaces	Yarn workspaces, Nx	Lightweight; works with pnpm; no magic

Environment Mgmt	.env files + dotenv	AWS Secrets Manager	Simple; works locally and in CI/CD; upgrade later
Logging	winston	pino, bunyan	Structured logs; multiple transports; financial audit trail ready
Background Jobs	node-cron (MVP)	Bull, RabbitMQ	Simple cron tasks suffice for MVP; upgrade to queue system later
API Documentation	OpenAPI (Swagger)	Blueprint, RAML	Industry standard; auto-generated; client code generation possible

2. ARCHITECTURE RECOMMENDATION: SEPARATE REPOSITORIES

Decision: Two Independent Repos (finpilot-backend & finpilot-frontend)

Rationale

- ✅ **Independent Deployment:** Deploy frontend & backend on separate schedules
- ✅ **Team Separation:** Different teams can work on frontend/backend independently
- ✅ **Clear Boundaries:** Explicit separation of concerns; API contract is the interface
- ✅ **Scalability:** Each repo can grow independently without monorepo overhead
- ✅ **Technology Freedom:** Can upgrade tech stack independently if needed later

Trade-offs vs. Monorepo

- ⚠️ **Type Sync:** Manual process to keep types in sync (use shared npm package)
- ⚠️ **API Contracts:** Need API documentation (OpenAPI/Swagger) + SDK generation
- ✅ **Deployment:** More flexibility (independent scales, deploys)
- ✅ **Team Autonomy:** Frontend can develop against mock API; backend can iterate independently

Type Sharing Strategy

- Backend publishes @finpilot/api-types npm package** (auto-generated from Zod schemas + Prisma)
- Frontend imports types** from the published package
- CI/CD** auto-publishes on main branch updates

Structure

finpilot-backend/ (this repo)

```
finpilot-backend/
├── src/
│   ├── api/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   ├── utils/
│   ├── config/
│   └── types/
├── prisma/
└── tests/
```

```
├─ .env.example
├─ package.json
├─ tsconfig.json
├─ jest.config.js
└─ README.md
```

finpilot-frontend/ (separate repo)

```
finpilot-frontend/
├─ src/
│  ├─ pages/
│  ├─ components/
│  ├─ hooks/
│  ├─ services/
│  ├─ store/
│  ├─ types/
│  ├─ utils/
│  ├─ styles/
│  └─ config/
├─ public/
├─ tests/
├─ .env.example
├─ package.json
├─ tsconfig.json
├─ vite.config.ts
└─ README.md
```

3. FOLDER STRUCTURE (DETAILED)

Backend Repository (finpilot-backend)

```
finpilot-backend/
├─ src/
│  ├─ main.ts                # Entry point
│  ├─ app.ts                 # Express app setup
│  ├─ server.ts              # HTTP server factory
│  │
│  └─ api/                   # HTTP layer
│     ├─ middleware/
│     │  ├─ auth.ts          # JWT verification
│     │  ├─ validation.ts    # Zod schema validation
│     │  ├─ errorHandler.ts  # Global error handling
│     │  ├─ logger.ts        # Request/response logging
│     │  └─ cors.ts          # CORS config
│     └─ routes/
│        ├─ index.ts         # Routes aggregator
│        ├─ auth.ts          # POST /auth/* endpoints
│        ├─ userRoutes.ts     # GET,PUT /users/* endpoints
│        ├─ transactionRoutes.ts
│        ├─ categoryRoutes.ts
│        └─ budgetRoutes.ts
```

```
├── dashboardRoutes.ts
├── reportRoutes.ts
├── syncRoutes.ts    # Bank sync mock
└── aiRoutes.ts      # AI endpoints

├── services/        # Business logic layer
│   ├── AuthService.ts
│   ├── TransactionService.ts
│   ├── BudgetService.ts
│   ├── CategoryService.ts
│   ├── DashboardService.ts
│   ├── CurrencyService.ts
│   ├── CsvImportService.ts
│   ├── ReportService.ts
│   ├── BankSyncService.ts
│   └── SavingsAIService.ts
├── repositories/    # Data access layer
│   ├── BaseRepository.ts    # Abstract base with common CRUD
│   ├── UserRepository.ts
│   ├── TransactionRepository.ts (special: filtering, aggregation)
│   ├── BudgetRepository.ts
│   ├── CategoryRepository.ts
│   ├── DashboardRepository.ts (aggregations)
│   └── index.ts
├── models/          # Prisma models go here (schema.prisma)
│   └── schema.prisma    # Single source of truth for DB
├── utils/
│   ├── validation.ts    # Custom validators
│   ├── currencyConverter.ts # Live currency conversions
│   ├── csvParser.ts     # CSV parsing + validation
│   ├── pdfGenerator.ts  # PDF creation
│   ├── jwt.ts           # JWT sign/verify
│   ├── password.ts      # Hash/compare passwords
│   └── logger.ts        # Winston logger singleton
├── config/
│   ├── env.ts           # Environment variables (validated)
│   ├── database.ts      # Prisma client + connection
│   ├── constants.ts     # App-wide constants
│   ├── seedCategories.ts # Default category data
│   └── index.ts
├── types/
│   ├── index.ts         # Local backend types
│   ├── api.ts           # Request/Response types (exported for frontend)
│   ├── models.ts        # Entity types
│   └── errors.ts        # Error types
└── prisma/
```

```

├── schema.prisma          # Prisma schema
├── seed.ts                # Seed script
├── migrations/            # Auto-generated by Prisma
├── tests/
│   ├── unit/
│   │   ├── services/
│   │   ├── repositories/
│   │   └── utils/
│   ├── integration/
│   │   ├── api.spec.ts
│   │   ├── services.spec.ts
│   │   └── fixtures.ts
│   └── setup.ts           # Jest config
├── .env.example
├── .env.test
├── .prettierrc
├── .eslinttrc.json
├── jest.config.js
├── package.json
├── tsconfig.json
└── README.md

```

Frontend Repository (`finpilot-frontend`)

```

finpilot-frontend/
├── src/
│   ├── main.tsx           # Entry point
│   ├── App.tsx            # Root component
│   ├──
│   └── pages/              # Page components (one per route)
│       ├── LoginPage.tsx
│       ├── DashboardPage.tsx
│       ├── TransactionPage.tsx
│       ├── BudgetPage.tsx
│       ├── ReportPage.tsx
│       ├── SettingsPage.tsx
│       └── NotFoundPage.tsx
│   ├── components/         # Reusable components
│   │   ├── Layout/
│   │   │   ├── MainLayout.tsx  # Header, sidebar wrapper
│   │   │   ├── Navbar.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   └── ProtectedRoute.tsx
│   │   ├── Forms/
│   │   │   ├── TransactionForm.tsx
│   │   │   ├── BudgetForm.tsx
│   │   │   ├── LoginForm.tsx
│   │   │   └── CsvUploadForm.tsx

```

- └─ Tables/
 - └─ TransactionTable.tsx
 - └─ BudgetTable.tsx
 - └─ hooks/
 - └─ useTableSort.ts
- └─ Charts/
 - └─ IncomeExpenseChart.tsx
 - └─ CategoryBreakdownPie.tsx
 - └─ BudgetProgressBar.tsx
 - └─ MonthlyTrendLine.tsx
 - └─ hooks/
 - └─ useChartTheme.ts
- └─ Cards/
 - └─ MetricCard.tsx # Reusable metric card
 - └─ BudgetCard.tsx
 - └─ StatCard.tsx
- └─ Inputs/
 - └─ CurrencyInput.tsx
 - └─ DateRangePicker.tsx
 - └─ CategorySelect.tsx
 - └─ Button.tsx # Custom button (Tailwind)
- └─ Modals/
 - └─ ConfirmDialog.tsx
 - └─ EditTransactionModal.tsx
 - └─ ErrorModal.tsx
- └─ Loading/
 - └─ Skeleton.tsx
 - └─ Spinner.tsx
- └─ hooks/ # Custom React hooks
 - └─ useAuth.ts # Auth context + login/logout
 - └─ useTransactions.ts # TanStack Query interaction
 - └─ useBudgets.ts
 - └─ useCategories.ts
 - └─ useDashboard.ts
 - └─ useCurrency.ts # User's base currency
 - └─ useDateRange.ts # Date filter state
 - └─ useNotification.ts # Toast/alert state
 - └─ usePagination.ts
- └─ services/
 - └─ api.ts # Axios client instance
 - └─ apiClient.ts # API routes (type-safe, using @finpilot/api-types)
 - └─ auth.ts # Token storage, refresh logic
 - └─ storage.ts # localStorage helpers
- └─ store/ # Global state (Zustand)
 - └─ index.ts # Root store
 - └─ authStore.ts # User, token
 - └─ uiStore.ts # Notifications, modals, filters
 - └─ cacheStore.ts # Local cache for offline

```

├── types/
│   ├── index.ts           # Re-export from @finpilot/api-types
│   ├── ui.ts             # UI component prop types
│   └── local.ts          # Frontend-only types
├── utils/
│   ├── formatters.ts     # Date, currency formatting
│   ├── calculations.ts   # Budget vs actual, percentages
│   ├── validation.ts     # Form validation helpers
│   ├── api.ts            # API call helpers
│   └── errorHandler.ts   # Error message parsing
├── styles/
│   ├── globals.css       # Tailwind imports, base styles
│   ├── variables.css     # CSS custom properties
│   └── components.css    # Component-scoped styles if needed
├── config/
│   ├── routes.ts         # Route definitions (centralized)
│   ├── api.ts            # API base URL, timeouts (load from env)
│   └── constants.ts      # App-wide constants
├── public/
│   ├── index.html (entry point)
│   └── favicon.ico
├── tests/
│   ├── unit/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── utils/
│   ├── integration/
│   ├── fixtures/
│   └── setup.ts
├── .env.example
├── .prettierrc
├── .eslintrc.json
├── package.json
├── tsconfig.json
├── vite.config.ts
├── vitest.config.ts
└── README.md

```

4. DATABASE SCHEMA

Entities & Relationships

```

-- Users
CREATE TABLE users (

```



```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
email VARCHAR(255) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,
name VARCHAR(255),
base_currency VARCHAR(3) DEFAULT 'USD',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
deleted_at TIMESTAMP NULL -- soft delete
);

-- Categories (with nested structure via parent_id)
CREATE TABLE categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name VARCHAR(100) NOT NULL,
  parent_id UUID REFERENCES categories(id) ON DELETE SET NULL,
  category_type ENUM('INCOME', 'EXPENSE') NOT NULL,
  color VARCHAR(7) DEFAULT '#3B82F6',
  icon VARCHAR(50),
  is_default BOOLEAN DEFAULT false,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(user_id, name, parent_id)
);

-- Transactions
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category_id UUID NOT NULL REFERENCES categories(id),
  description VARCHAR(500),
  amount DECIMAL(15, 2) NOT NULL, -- original currency
  base_currency_amount DECIMAL(15, 2) NOT NULL, -- converted to user's base
currency
  currency VARCHAR(3) NOT NULL,
  exchange_rate DECIMAL(10, 6), -- rate used at write time
  transaction_date DATE NOT NULL,
  transaction_type ENUM('INCOME', 'EXPENSE') NOT NULL,
  tags JSONB DEFAULT '[]', -- flexible tagging
  notes TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_date (user_id, transaction_date),
  INDEX idx_user_category (user_id, category_id),
  INDEX idx_user_type (user_id, transaction_type)
);

-- Budgets (per category, per month)
CREATE TABLE budgets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category_id UUID NOT NULL REFERENCES categories(id),

```

```

amount DECIMAL(15, 2) NOT NULL, -- in base currency
month DATE NOT NULL, -- first day of month (YYYY-MM-01)
alert_threshold DECIMAL(3, 2) DEFAULT 0.9, -- alert at 90%
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
UNIQUE(user_id, category_id, month)
);

-- ExchangeRates (for historical accuracy)
CREATE TABLE exchange_rates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  base_currency VARCHAR(3) NOT NULL,
  target_currency VARCHAR(3) NOT NULL,
  rate DECIMAL(10, 6) NOT NULL,
  source VARCHAR(50) DEFAULT 'FIXED', -- FIXED, ECB, FIXER, etc.
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(base_currency, target_currency, date(updated_at))
);

-- BankSync logs
CREATE TABLE bank_sync_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  synced_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  transaction_count INT,
  status ENUM('SUCCESS', 'FAILURE', 'PARTIAL'),
  error_message TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- AuditLog (for compliance)
CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE SET NULL,
  entity_type VARCHAR(50), -- 'TRANSACTION', 'BUDGET', etc.
  entity_id UUID,
  action VARCHAR(50), -- CREATE, UPDATE, DELETE, EXPORT
  old_values JSONB,
  new_values JSONB,
  ip_address INET,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_entity (user_id, entity_type)
);

```

Prisma Schema (equivalent)

```

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  passwordHash String
  name        String?

```

```

    baseCurrency      String      @default("USD")
    createdAt         DateTime    @default(now())
    updatedAt         DateTime    @updatedAt
    deletedAt         DateTime?

    transactions      Transaction[]
    categories         Category[]
    budgets            Budget[]
    bankSyncLogs       BankSyncLog[]
    auditLogs          AuditLog[]
}

model Category {
  id                String      @id @default(cuid())
  userId            String
  user              User        @relation(fields: [userId], references: [id],
onDelete: Cascade)
  name              String
  parentId          String?
  parent            Category?   @relation("CategoryHierarchy", fields: [parentId],
references: [id], onDelete: SetNull)
  children           Category[] @relation("CategoryHierarchy")
  categoryType       String      // 'INCOME' | 'EXPENSE'
  color              String      @default("#3B82F6")
  icon              String?
  isDefault          Boolean      @default(false)
  createdAt          DateTime    @default(now())
  updatedAt          DateTime    @updatedAt

  transactions      Transaction[]
  budgets            Budget[]

  @@unique([userId, name, parentId])
  @@index([userId])
}

model Transaction {
  id                String      @id @default(cuid())
  userId            String
  user              User        @relation(fields: [userId], references: [id],
onDelete: Cascade)
  categoryId         String
  category           Category    @relation(fields: [categoryId], references: [id])
  description        String?
  amount             Decimal      @db.Decimal(15, 2)
  baseCurrencyAmount Decimal      @db.Decimal(15, 2)
  currency           String
  exchangeRate        Decimal?    @db.Decimal(10, 6)
  transactionDate    DateTime    @db.Date
  transactionType     String      // 'INCOME' | 'EXPENSE'
  tags               String[]     @default([])
  notes              String?

```

```

    createdAt      DateTime      @default(now())
    updatedAt      DateTime      @updatedAt

    @@index([userId, transactionDate])
    @@index([userId, categoryId])
    @@index([userId, transactionType])
}

model Budget {
  id              String          @id @default(cuid())
  userId          String
  user            User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
  categoryId      String
  category        Category        @relation(fields: [categoryId], references: [id])
  amount          Decimal          @db.Decimal(15, 2)
  month           DateTime          @db.Date // first of month
  alertThreshold  Decimal          @default(0.9) @db.Decimal(3, 2)
  createdAt       DateTime          @default(now())
  updatedAt       DateTime          @updatedAt

  @@unique([userId, categoryId, month])
  @@index([userId, month])
}

model ExchangeRate {
  id              String          @id @default(cuid())
  baseCurrency    String
  targetCurrency  String
  rate            Decimal          @db.Decimal(10, 6)
  source          String          @default("FIXED")
  updatedAt       DateTime          @updatedAt

  @@unique([baseCurrency, targetCurrency, updatedAt])
  @@index([baseCurrency])
}

model BankSyncLog {
  id              String          @id @default(cuid())
  userId          String
  user            User            @relation(fields: [userId], references: [id],
onDelete: Cascade)
  syncedAt        DateTime          @default(now())
  transactionCount Int?
  status           String          // 'SUCCESS' | 'FAILURE' | 'PARTIAL'
  errorMessage     String?
  createdAt       DateTime          @default(now())

  @@index([userId])
}

model AuditLog {

```

```

    id                String          @id @default(cuid())
    userId            String?
    user              User?           @relation(fields: [userId], references: [id],
onDelete: SetNull)
    entityType        String
    entityId           String?
    action            String
    oldValues          Json?
    newValues          Json?
    ipAddress          String?
    createdAt          DateTime       @default(now())

    @@index([userId, entityType])
}

```

Key Schema Decisions

1. **baseCurrencyAmount stored at write time** → performance, historical accuracy
2. **Soft deletes on User** → audit trail preservation
3. **Category hierarchy via parent_id** → flexible subcategories without extra tables
4. **Exchange rates table** → historical tracking, audit compliance
5. **AuditLog table** → financial compliance (who did what, when)
6. **JSONB tags on Transaction** → flexibility, extensible system

5. API ENDPOINT LIST

All endpoints require authentication via Bearer token in `Authorization` header (except `/auth/*` and `/health`).

Authentication

POST	<code>/auth/register</code>	Register new user
POST	<code>/auth/login</code>	Login, receive JWT
POST	<code>/auth/refresh</code>	Refresh expired token
POST	<code>/auth/logout</code>	Invalidate token (optional, tokens are stateless)

Users

GET	<code>/users/me</code>	Get current user profile
PUT	<code>/users/me</code>	Update user (name, base currency)
DELETE	<code>/users/me</code>	Delete account (soft delete)

Transactions

GET	<code>/transactions</code>	List transactions (paginated, filterable)
-----	----------------------------	---

Query params:

- `limit=20` (default), `offset=0`
- `dateFrom=YYYY-MM-DD`
- `dateTo=YYYY-MM-DD`

- categoryId=<uuid>
- type=INCOME|EXPENSE
- search=<string> (description, tags)

POST	/transactions	Create new transaction
GET	/transactions/:id	Get single transaction
PUT	/transactions/:id	Update transaction
DELETE	/transactions/:id	Delete transaction
POST	/transactions/bulk-import	Import CSV (multipart form)
GET	/transactions/export	Export to CSV (static snapshot)

Categories

GET	/categories	List all categories for user
POST	/categories	Create category (with optional parent_id)
PUT	/categories/:id	Update category
DELETE	/categories/:id	Delete category (if no transactions)
GET	/categories/:id/subcategories	Get child categories

Budgets

GET	/budgets	List budgets (by month or all-time)
Query params:		
- month=YYYY-MM		
- categoryId=<uuid>		
POST	/budgets	Create budget
PUT	/budgets/:id	Update budget
DELETE	/budgets/:id	Delete budget
GET	/budgets/current-month	Get all budgets for current month
GET	/budgets/:id/actual-spending	Get actual spending vs budget for period

Dashboard

GET	/dashboard/summary	Get dashboard KPIs
Returns:		
{		
totalIncome: number,		
totalExpense: number,		
balance: number,		
savingsRate: number,		
currentMonth: { income, expense, target: number },		
topExpenseCategories: [{ name, amount, percentage }]		
}		
GET	/dashboard/trends	Get monthly trends (6-12 months)
Returns: [{ month, income, expense, balance }]		

```
GET    /dashboard/budget-overview    Get budget status for all categories
Returns: [ { categoryId, name, budgeted, spent, remaining, percentageUsed } ]
```

Reports

```
POST    /reports/generate    Generate PDF report
Body:
{
  dateFrom: "YYYY-MM-DD",
  dateTo: "YYYY-MM-DD",
  includeBudgets: boolean,
  includeCharts: boolean,
  format: "PDF" | "CSV"
}
Returns: { reportId, downloadUrl }
```

```
GET    /reports/:id    Download generated report
```

Bank Sync (Mock)

```
POST    /sync/bank    Simulate bank sync
Returns:
{
  syncedAt: ISO string,
  transactionCount: number,
  newTransactions: [ Transaction[] ],
  status: "SUCCESS" | "PARTIAL"
}
```

AI Endpoints

```
GET    /ai/savings-recommendations    Get AI savings insights
Query params:
- months=6 (rolling period)
Returns:
{
  recommendations: [
    {
      category: "Dining Out",
      currentSpend: 450,
      historicalAvg: 420,
      potentialSavings: 100,
      insight: "Your dining spending is 7% above average..."
    }
  ],
  overallInsight: "..."
}
```

```
GET    /ai/budget-forecast    AI forecast for next month
Returns:
```

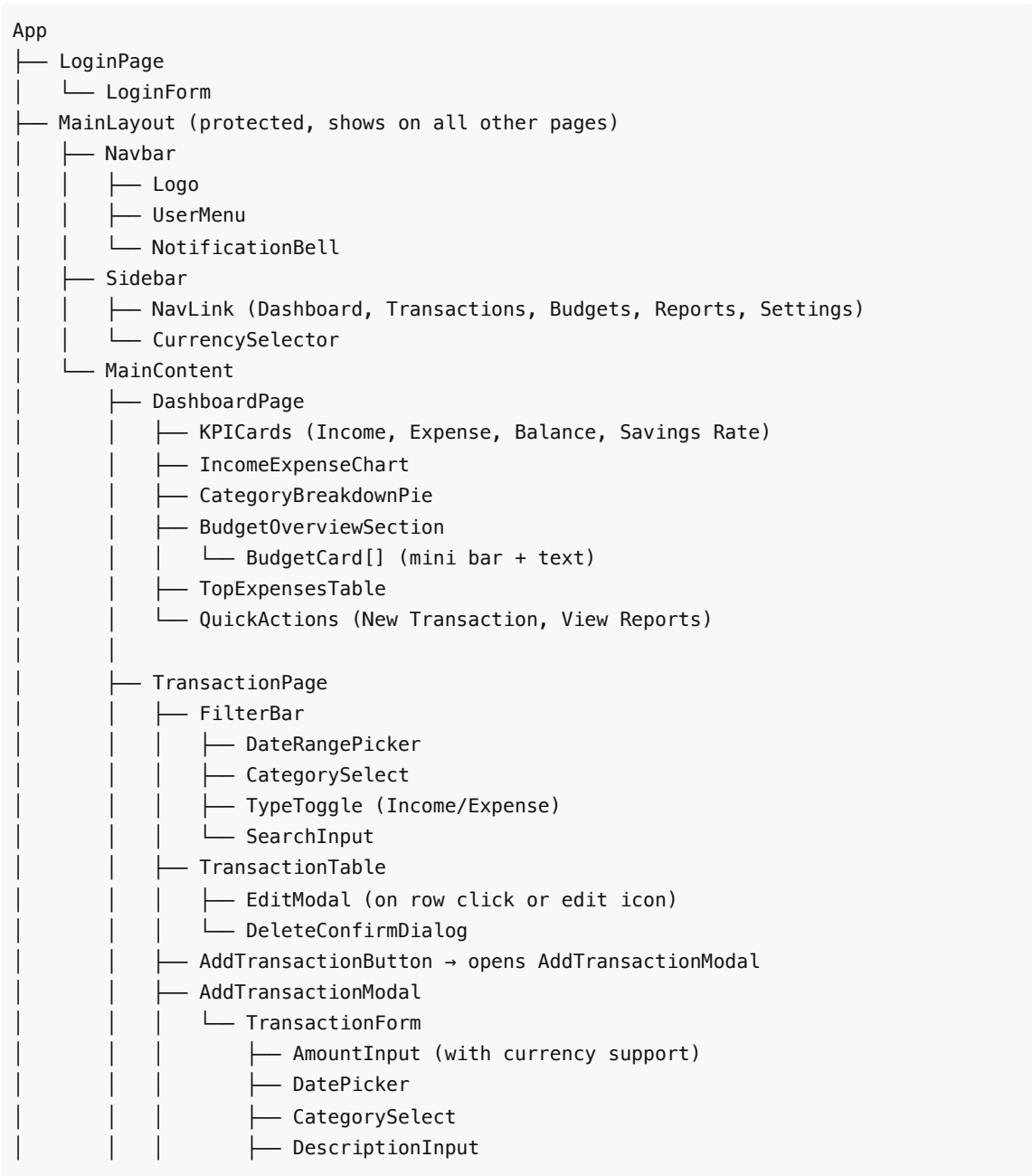
```
{
  predictedCategories: [ { category, forecastAmount, confidence } ]
}
```

Health / Monitoring

GET	/health	Health check (no auth required)
GET	/health/ready	Readiness probe (DB connectivity)

6. FRONTEND PAGES & COMPONENTS

Page Hierarchy




```
    └─ SubmitButton
  └─ CsvUploadButton → opens CsvUploadModal
    └─ CsvUploadForm
      └─ FileInput
      └─ ValidationResults
      └─ ImportButton

  └─ BudgetPage
    └─ MonthSelector (prev/next month)
    └─ AddBudgetButton → opens SetBudgetModal
    └─ BudgetList
      └─ BudgetCard[]
        └─ CategoryName
        └─ ProgressBar (percentage spent)
        └─ Text (Budgeted: $X, Spent: $Y, Remaining: $Z)
        └─ Alert (if over budget)
        └─ EditButton/DeleteButton
    └─ SummaryStats (Total Budgeted, Total Spent, Total Remaining)
    └─ SetBudgetModal
      └─ BudgetForm
        └─ CategorySelect
        └─ AmountInput
        └─ SubmitButton

  └─ ReportPage
    └─ ReportForm
      └─ DateRangePickerSection
      └─ CheckboxGroup (Include Budgets, Charts, etc.)
      └─ FormatToggle (PDF / CSV)
    └─ GenerateButton → triggers API call
    └─ RecentReports
      └─ ReportCard[]
        └─ DateRange
        └─ GeneratedAt
        └─ DownloadButton

  └─ SettingsPage
    └─ UserSettings
      └─ NameInput
      └─ EmailDisplay (read-only)
      └─ CurrencySelect
      └─ SaveButton
    └─ SecuritySettings
      └─ ChangePasswordButton → Modal with OldPassword + NewPassword
      └─ LogoutAllDevices
    └─ BankSyncSettings
      └─ LastSyncedAtDisplay
      └─ SyncNowButton
      └─ SyncStatusIndicator
    └─ DangerZone
      └─ DeleteAccountButton → ConfirmDialog
```

Reusable Component Library

Forms

- `TransactionForm` - controlled inputs + validation
- `BudgetForm` - amount input + category select
- `LoginForm` - email + password
- `CsvUploadForm` - file upload + progress

Tables

- `TransactionTable` - sortable, paginated, filterable, with inline edit
- `BudgetTable` - category, amount, spent, remaining, progress bar

Charts (using Recharts)

- `IncomeExpenseChart` - bar/line chart by month
- `CategoryBreakdownPie` - pie chart of top categories
- `BudgetProgressBar` - horizontal bar showing usage
- `MonthlyTrendLine` - line chart of balance over time

Cards

- `MetricCard` - icon + title + value (Income, Expense, etc.)
- `BudgetCard` - category + progress + amount
- `StatCard` - flexible layout for KPIs

Inputs

- `CurrencyInput` - text input with currency prefix/symbol
- `DateRangePicker` - from/to date selection
- `CategorySelect` - searchable dropdown
- `AmountInput` - numeric with formatting
- `Button` - Tailwind-styled variants (primary, secondary, danger)

Modals

- `ConfirmDialog` - yes/no confirmation
- `EditTransactionModal` - full transaction form in modal
- `ErrorModal` - error message display

Loading

- `Skeleton` - content placeholder
- `Spinner` - loading indicator

7. SERVICE BOUNDARIES (Backend)

Layer Responsibilities

API Layer (`api/routes` , `api/middleware`)

- Accept HTTP requests

- Parse & validate input (using Zod schemas)
- Call service methods
- Format & return JSON responses
- Handle auth middleware

Service Layer (`services/`)

- **AuthService:** Signup, login, token refresh, password validation
- **TransactionService:** CRUD, filtering, aggregations (sum by category), bulk import
- **BudgetService:** CRUD budgets, calculate spent vs budget, budget alerts
- **CategoryService:** CRUD categories, manage hierarchy
- **DashboardService:** Aggregate totals, trends, top categories (orchestrates repos)
- **CurrencyService:** Convert amounts, validate currencies, cache rates
- **CsvImportService:** Parse CSV, validate rows, transform to transactions, bulk insert
- **ReportService:** Generate PDF, aggregate data for report period
- **BankSyncService:** Simulate bank API call, create transactions, log sync
- **SavingsAIService:** Aggregate 6-month summaries, call external AI, parse recommendations

Repository Layer (`repositories/`)

- Query builders (find, findMany, create, update, delete)
- Complex queries (aggregations, joins, nested filtering)
- No business logic
- "Dumb" data access

Specific Repositories:

- **BaseRepository:** Abstract base for CRUD operations
- **TransactionRepository:** Special filtering (by date range, category, type), aggregations
- **BudgetRepository:** Month-based queries, spent calculation (via aggregation)
- **DashboardRepository:** Pre-built aggregation queries (total income, top categories, etc.)

Models (`models/schema.prisma`)

- Database schema definition
- Prisma generates `@prisma/client` for type-safe queries

Utils (`utils/`)

- **validation.ts:** Zod schema definitions (reuse in API validation)
- **currencyConverter.ts:** Convert amounts, fixed rates
- **csvParser.ts:** Parse CSV file into transaction objects
- **pdfGenerator.ts:** Build PDF structure, write to stream
- **jwt.ts:** Sign & verify JWT tokens
- **password.ts:** Hash & compare passwords
- **logger.ts:** Structured logging (Winston)

Config (`config/`)

- Environment variables (validated via Zod)
- Prisma client instance
- App constants
- Seed data

8. TRADEOFFS FOR KEY DECISIONS

Decision 1: Monorepo vs. Separate Repos

Aspect	Monorepo	Separate Repos
Type Safety	✔ Shared types auto-sync	✗ Manual duplication
Development Speed	✔ One <code>pnpm install</code>	✗ Two installations
API Contracts	✔ Single source of truth	✗ Version mismatch risk
Deployment	✔ Atomic	⚠ Coordination needed
Team Workflows	✔ Full-stack commits	✗ Split ownership
Scalability	⚠ Works until ~1M LOC	✔ No limit
Initial Complexity	✔ Simple setup	✗ Multiple repos

Decision: Separate Repos (✔ independent deployment, team autonomy, clear API contract)

Decision 2: Base Currency Conversion Timing

Aspect	At Write Time	At Read Time
Performance	✔ O(1) aggregations	✗ Slow aggregations
Historical Accuracy	✔ Rate never changes	✗ Retroactive changes if rate updates
Audit Trail	✔ Original rate stored	✗ Lost forever
Complexity	⚠ Extra column	✔ Simpler schema
Financial Auditing	✔ Perfect	✗ Gaps

Decision: At Write Time (✔ financial transactions need this)

Decision 3: Dashboard Aggregation Caching

Aspect	Computed On-Demand	Pre-computed Cache
Freshness	✔ Always latest	✗ Stale if infrequent updates
Performance	✗ DB hit per request	✔ Cache hit, instant
Infrastructure	✔ Simpler	✗ Cache layer needed
Scalability	✗ Gets slow	✔ Scales to many users
MVP Suitability	✔ Good enough	✗ Over-engineered

Decision: Computed On-Demand (✔ for MVP, add caching/materialized views later)

Decision 4: AI Input: Raw Transactions vs. Aggregated Summary

Aspect	Raw Transactions	Pre-aggregated
--------	------------------	----------------

Security	✗ Detailed txn data exposed	✓ Only summaries
Privacy	✗ Could reveal purchasing patterns	✓ Stripped of details
Cost	✗ More data = more API calls	✓ Smaller payloads
AI Quality	✓ More data	⚠ Might suffice
Compliance	✗ PII in transit	✓ Safe

Decision: Pre-aggregated Summary (✓ privacy & compliance first)

Example payload:

```
{
  "userId": "hash-not-real-id",
  "months": [
    {
      "month": "2026-03",
      "categories": { "groceries": 450, "dining": 200 },
      "totalIncome": 5000,
      "totalExpense": 1800
    },
    {
      "month": "2026-02",
      "categories": { "groceries": 480, "dining": 250 },
      "totalIncome": 5000,
      "totalExpense": 1900
    }
  ]
}
```

Decision 5: Authentication Strategy

Aspect	JWT + HttpOnly Cookies	Session + Redis
Statefulness	✓ Stateless	✗ Server state needed
Scalability	✓ No DB lookups	✗ Redis required
Mobile Support	⚠ Token in storage	✓ Cookies auto-sent
CSRF Protection	✓ Built-in (same-site)	✓ CSRF tokens
Token Refresh	⚠ Extra endpoint	✓ Transparent
Logout	⚠ Doesn't invalidate	✓ Instant revocation

Decision: JWT + HttpOnly Cookies (✓ stateless, CORS-friendly, scales)

Decision 6: Form State Management

Aspect	TanStack Query	Zustand	React Hook Form
--------	----------------	---------	-----------------

Server State	✅ Designed for	❌ Not ideal	⚠️ Hybrid
Form State	❌ Overkill	⚠️ Manual	✅ Designed for
Caching	✅ Automatic	❌ Manual	❌ No
Bundle Size	🔴 12KB	🟢 2KB	🟢 3KB
Learning Curve	⚠️ Moderate	✅ Shallow	✅ Shallow

Decision: React Hook Form + Zod for forms, TanStack Query for server state, Zustand for UI state only

Decision 7: Component Library vs. Build-Your-Own

Aspect	Shadcn/ui (Headless UI)	Material-UI	Tailwind only
Customization	✅ Full	⚠️ Limited	✅ Full
Bundle Size	🟢 Small	🔴 Large	🟢 Tailwind
Accessibility	✅ Built-in	✅ Built-in	⚠️ Manual
Speed	✅ Fast	❌ Slower	✅ Fast
Design System	⚠️ Minimal	✅ Opinionated	⚠️ Manual

Decision: Headless UI + Tailwind (✅ lightweight, accessible, customizable)

Decision 8: Validation Library

Aspect	Zod	Joi	Yup
Schema Language	✅ TypeScript DSL	⚠️ Object DSL	⚠️ Object DSL
Type Inference	✅ Perfect	⚠️ Manual	⚠️ Manual
Bundle Size	🟢 Small	🔴 Medium	🟢 Small
Learning Curve	✅ Steep	⚠️ Moderate	✅ Easy
Backend & Frontend	✅ Works both	✅ Node only	✅ Works both

Decision: Zod (✅ shared with backend, perfect type inference)

Decision 9: CSV Upload Handling

Approach	Streaming	Load All in Memory	Database Stream
Performance	✅ Fast	❌ Slow for large	⚠️ Medium
Memory Usage	✅ Constant	❌ O(n)	⚠️ O(batch)
Error Recovery	⚠️ Hard	✅ Easy	✅ Can rollback
Complexity	❌ Complex	✅ Simple	⚠️ Moderate

MVP Suitability	❌ Overkill	✅ Simple	⚠️ Good middle
-----------------	------------	----------	----------------

Decision: Database Stream + Transaction Rollback (✅ balance of simplicity & robustness)

Decision 10: PDF Generation Library

Aspect	pdfkit	jsPDF	puppeteer
Type Safety	⚠️ No	❌ No	❌ No
Performance	✅ Fast	✅ Fast	❌ Slow (headless)
PDF Quality	✅ Good	✅ Good	✅ Perfect (renders HTML)
Server Resource	✅ Light	✅ Light	❌ Heavy (browser)
Simplicity	✅ Simple	⚠️ Moderate	❌ Complex
MVP Suitability	✅ Yes	✅ Yes	❌ Overkill

Decision: pdfkit (✅ lightweight, good quality, easy setup)

Summary of Key Architecture Principles

- API-First Design:** Frontend and backend communicate via REST; API contract is the interface
- Type Sharing via npm Package:** Backend publishes `@finpilot/api-types` for frontend consumption
- Independent Deployment:** Each repo deploys separately; loosely coupled, independently scalable
- Service Layer Isolation:** Business logic in services, not routes or repositories
- Financial Accuracy:** Base currency amount stored at write time; audit trail maintained
- Privacy by Default:** AI receives aggregated data, never raw transactions
- MVP Speed Over Perfection:** Dashboard on-demand, simple auth, no overengineering
- Type Safety Everywhere:** TypeScript, Zod schemas, Prisma — no JSON holes (auto-generated)
- Testability:** Services depend on repos; repos testable in isolation
- Scalability Path:** Design allows for caching, queue jobs, materialized views later

9. IMPLEMENTATION ORDER

Phase 0 — Project Scaffold

Goal: Two repos standing, compiling, and runnable (empty shells)

Step	Task	Output
0.1	Init finpilot-backend — npm init, TypeScript, ESLint, Prettier	Compiles with tsc
0.2	Init finpilot-frontend — Vite + React + TS + Tailwind	Runs with npm run dev
0.3	Configure .env.example, .gitignore, README stubs	Documented setup
0.4	Set up PostgreSQL locally (or Docker)	DB accessible

0.5	Init Prisma, create schema.prisma with all 7 entities	npx prisma migrate dev succeeds
0.6	Seed script — default categories, test user	Database populated

Phase 1 — Auth + User

Goal: Register, login, protect routes

Step	Task	Output
1.1	Express app setup (app.ts, server.ts, main.ts)	Server starts on port 3000
1.2	Middleware: errorHandler, cors, logger, validation	Foundation layer ready
1.3	UserRepository + AuthService (register, login, refresh)	Service layer working
1.4	Auth routes (/auth/register, /auth/login, /auth/refresh)	Postman-testable
1.5	Auth middleware (JWT verify)	Protected routes reject unauthenticated
1.6	User routes (/users/me GET, PUT)	Profile readable/updatable

Phase 2 — Categories + Transactions CRUD

Goal: Core financial data flowing end-to-end

Step	Task	Output
2.1	CategoryRepository + CategoryService	CRUD for categories
2.2	Category routes (GET, POST, PUT, DELETE)	Postman-testable
2.3	CurrencyService — fixed exchange rates, convert function	Conversion logic isolated
2.4	TransactionRepository + TransactionService	CRUD + baseCurrencyAmount at write time
2.5	Transaction routes (full CRUD + pagination + filtering)	Filterable list working
2.6	Zod schemas for all inputs (shared validation)	Input validation on all routes

Phase 3 — Frontend Foundation

Goal: Login working, transactions visible and editable

Step	Task	Output
------	------	--------

3.1	Axios API client + auth interceptor	Auto-attaches JWT, handles refresh
3.2	Zustand auth store + useAuth hook	Login state managed
3.3	LoginPage + LoginForm	User can register/login
3.4	MainLayout + Sidebar + Navbar + ProtectedRoute	App shell with navigation
3.5	TransactionPage + TransactionTable + TransactionForm	Full transaction CRUD in UI
3.6	useTransactions hook (TanStack Query)	Data fetching with caching

Phase 4 — Dashboard + Budgets

Goal: Financial overview and budget tracking

Step	Task	Output
4.1	DashboardRepository + DashboardService — aggregation queries	Totals, trends, top categories
4.2	Dashboard routes (/dashboard/summary, /dashboard/trends)	JSON KPIs returned
4.3	DashboardPage + MetricCard + IncomeExpenseChart + CategoryBreakdownPie	Visual dashboard
4.4	BudgetRepository + BudgetService (CRUD + actual vs budgeted)	Budget logic working
4.5	Budget routes (full CRUD + /budgets/current-month)	Postman-testable
4.6	BudgetPage + BudgetCard + BudgetProgressBar	Budget tracking in UI
4.7	useDashboard, useBudgets hooks	Frontend data layer done

Phase 5 — CSV Import + Reports

Goal: Bulk data in, formatted data out

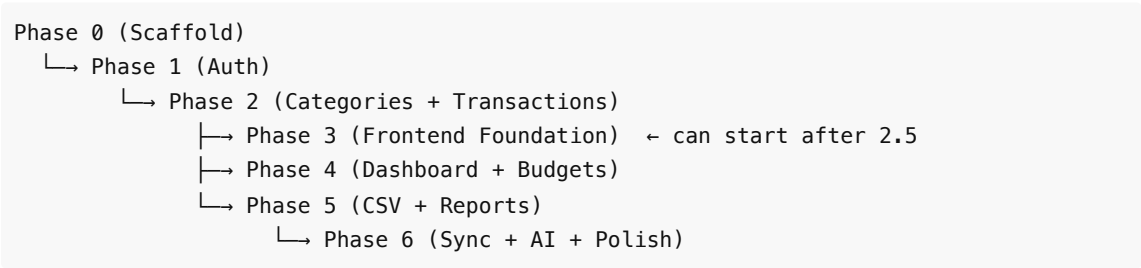
Step	Task	Output
5.1	CsvImportService — parse, validate, bulk insert with rollback	CSV processed correctly
5.2	POST /transactions/bulk-import route (multipart)	CSV upload via API
5.3	CsvUploadForm component + validation feedback UI	User uploads CSV in browser
5.4	ReportService + pdfGenerator utility	PDF generated from date range
5.5	Report routes (POST /reports/generate, GET /reports/:id)	PDF downloadable
5.6	ReportPage + report form + download UI	End-to-end report generation
5.7	GET /transactions/export — CSV export	Data export working

Phase 6 — Bank Sync + AI + Polish

Goal: Mock integrations, AI recommendations, production readiness

Step	Task	Output
6.1	BankSyncService — mock endpoint returning fake transactions	Simulated bank sync
6.2	POST /sync/bank route + sync log table updates	Sync logged
6.3	Bank sync UI in Settings page	User triggers sync from UI
6.4	SavingsAIService — aggregate summaries, call AI, parse response	AI integration working
6.5	AI routes (/ai/savings-recommendations, /ai/budget-forecast)	Recommendations returned
6.6	AI insights widget on DashboardPage	Recommendations visible
6.7	AuditLog writes on all CUD operations	Compliance trail complete
6.8	SettingsPage (profile, currency, password change, danger zone)	Settings functional
6.9	Error states, loading skeletons, 404 page, final polish	Production-ready UX

Implementation Dependency Graph



Vertical Slice Priority (per PROJECT_CONTEXT.md)

"Build a working vertical slice first, then expand."

The vertical slice is: **Phase 0 → 1 → 2 → 3**
After that, a user can register, log in, create transactions, and see them listed.
Everything else builds on top of that working core.